

重慶理工大學

实验报告书

实验名称：项目一 SQL 语句实践

班　　级 _____

学　　号 _____

姓　　名 _____

指导老师 _____

实验时间 _____

计算机科学与工程学院　计算机实验室（中心）

实验一　数据定义

一、学时

2 学时。

二、实验类型

设计。

三、实验目的

1. 巩固数据库基础知识，熟悉 MySQL 常用数据类型。
2. 掌握数据表的建立、修改和删除方法。
3. 掌握主码、外码、唯一约束和检查约束的定义与验证方法。
4. 使用 SQL 创建索引并进行相关操作。

四、实验准备

1. 熟悉 SQL Server 的工作环境。
2. 复习有关表的建立、修改和删除的 SQL 语言命令和方法。
3. 复习有关索引的建立、修改和删除的 SQL 语言命令和方法。

五、实验方法及步骤

1. 登录企业管理器 / 查询分析器。
2. 创建数据库或连接已建立的数据库。
3. 在当前数据库上建立新表，并注意定义表的主码、外码。
4. 修改表的结构。
5. 在表上建立索引。
6. 删除表的定义。

六、实验结果

1. 创建校园超市数据库

```
CREATE DATABASE SuperMarket
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE SuperMarket;
```

执行结果与分析：指定 utf8mb4 字符集以保存中文；排序规则采用 utf8mb4_0900_ai_ci。执行后当前数据库为 SuperMarket。

2. 创建商品关系表 Goods

```
CREATE TABLE Goods (
  GoodsNO VARCHAR(8) NOT NULL,
  SupplierNO VARCHAR(8) NOT NULL,
  CategoryNO VARCHAR(8) NOT NULL,
  GoodsName VARCHAR(60) NOT NULL,
  InPrice FLOAT NOT NULL,
  SalePrice DECIMAL(10,2) NOT NULL,
  Number INT NOT NULL,
  ProductTime DATE NOT NULL,
  CONSTRAINT pk_goods PRIMARY KEY (GoodsNO)
) ENGINE=InnoDB;
```

执行结果与分析：商品编码为主码。进价先定义为 FLOAT，供第 11 项演示类型修改；售价用 DECIMAL，生产时间用 DATE。由于被引用表尚未创建，供应商和类别外键在第 6 项补齐。执行成功，初始表包含 8 个属性列。

3. 创建学生关系表 Student

```
CREATE TABLE Student (
  SNO VARCHAR(8) NOT NULL,
  SName VARCHAR(30) NOT NULL,
  BirthYear SMALLINT NOT NULL,
  Gender VARCHAR(2) NOT NULL,
  College VARCHAR(10) NOT NULL,
  Major VARCHAR(10) NOT NULL,
  WeiXin VARCHAR(40),
  CONSTRAINT pk_student PRIMARY KEY (SNO),
  CONSTRAINT uq_student_weixin UNIQUE (WeiXin)
) ENGINE=InnoDB;
```

执行结果与分析：出生年份采用 SMALLINT，学号为主码，微信号通过命名唯一约束限制重复。SHOW INDEX 显示 PRIMARY 和 uq_student_weixin 两个唯一索引。两名学生使用相同的非空微信号时，第二次插入返回 ERROR 1062。

4. 创建销售关系表 SaleBill

```
CREATE TABLE SaleBill (
  GoodsNO VARCHAR(8) NOT NULL,
  SNO VARCHAR(8) NOT NULL,
  HappenTime DATE NOT NULL,
  Number INT NOT NULL,
  CONSTRAINT pk_salebill PRIMARY KEY (GoodsNO, SNO),
  CONSTRAINT fk_salebill_goods FOREIGN KEY (GoodsNO)
    REFERENCES Goods (GoodsNO),
  CONSTRAINT fk_salebill_student FOREIGN KEY (SNO)
    REFERENCES Student (SNO)
) ENGINE=InnoDB;
```

执行结果与分析：商品编码与学号共同构成主码，分别引用 Goods 和 Student。SHOW CREATE TABLE 显示复合主码及两个外键。按照题目指定的主码，同一学生对同一商品不能保存多条销售记录，即使销售日期不同也会冲突。

5. 创建商品类别表 Category

```
CREATE TABLE Category (
  CategoryNO VARCHAR(8) NOT NULL,
  CategoryName VARCHAR(30) NOT NULL,
  Descriptions VARCHAR(200),
  CONSTRAINT pk_category PRIMARY KEY (CategoryNO)
) ENGINE=InnoDB;
```

执行结果与分析：类别编号为主码，类别名称为非空字符串，描述列允许为空。执行成功，表中包含 CategoryNO、CategoryName 和 Descriptions 三列。

6. 创建供应商表并补齐商品外键

```
CREATE TABLE Supplier (
  SupplierNO VARCHAR(8) NOT NULL,
  SupplierName VARCHAR(60) NOT NULL,
  Address VARCHAR(100) NOT NULL,
  Number VARCHAR(20) NOT NULL,
  CONSTRAINT pk_supplier PRIMARY KEY (SupplierNO)
) ENGINE=InnoDB;
ALTER TABLE Goods
  ADD CONSTRAINT fk_goods_supplier FOREIGN KEY (SupplierNO)
    REFERENCES Supplier (SupplierNO),
  ADD CONSTRAINT fk_goods_category FOREIGN KEY (CategoryNO)
    REFERENCES Category (CategoryNO);
```

执行结果与分析：供应商编号为主码。供应商的 Number 表示电话，使用 VARCHAR(20) 保存，避免作为整数处理而丢失前导零。两个父表建立后，通过 ALTER TABLE 为 Goods 补齐外键。元数据查询共返回 4 个外键关系。

关系表	外码属性	被引用主码
Goods	SupplierNO	Supplier.SupplierNO
Goods	CategoryNO	Category.CategoryNO
SaleBill	GoodsNO	Goods.GoodsNO
SaleBill	SNO	Student.SNO

7. 增加保质期属性列

```
ALTER TABLE Goods ADD COLUMN QGperiod INT;
```

执行结果与分析：使用 ADD COLUMN 增加整型 QGperiod。列允许为空，因为未设置保质期默认值。SHOW COLUMNS 返回类型 int、允许 NULL。

8. 删除出生年份属性列

```
ALTER TABLE Student DROP COLUMN BirthYear;
```

执行结果与分析：使用 DROP COLUMN 删除 BirthYear。SHOW COLUMNS 中不再出现该列，Student 剩余 6 个属性列。后续验证时不再导入出生年份；其它实验若需要年龄，应恢复原始结构。

9. 增加数量大于零的约束

```
ALTER TABLE SaleBill
  ADD CONSTRAINT chk_salebill_positive CHECK (Number > 0);
```

执行结果与分析：命名 CHECK 约束要求 Number 大于零。样本中的正数销售数量可写入，将已有记录的数量改为 0 时返回 ERROR 3819，说明约束实际生效。Number 同时保留 NOT NULL，防止空值绕过业务要求。

10. 删除微信号唯一约束

```
ALTER TABLE Student DROP INDEX uq_student_weixin;
```

执行结果与分析：MySQL 的 UNIQUE 约束通过唯一索引实现，此处使用 DROP INDEX 删除。Student 的 PRIMARY 保留；将 S02 的微信号改成与 S01 相同的 wx001 后执行成功，查询同微信号记录数为 2。

11. 修改进价为一位小数的精确数值型

```
ALTER TABLE Goods MODIFY COLUMN InPrice DECIMAL(10,1) NOT NULL;
```

执行结果与分析：修改为 DECIMAL(10,1)，总精度为 10 位、小数位为 1 位，同时保留 NOT NULL。SHOW COLUMNS 返回 decimal(10,1)。样本进价可正常保存；将 2.56 写入该列后实际保存为 2.6。

12. 将数量约束修改为零至一百

```
ALTER TABLE SaleBill
  DROP CHECK chk_salebill_positive,
  ADD CONSTRAINT chk_salebill_range CHECK (Number BETWEEN 0 AND 100);
```

执行结果与分析：题目中的 0 ~ 100 按包含两个端点理解。MySQL 中先删除旧 CHECK，再添加新 CHECK；两个动作在同一个 ALTER TABLE 语句中完成。旧名称不再出现，新的 chk_salebill_range 实际约束为 Number BETWEEN 0 AND 100。

数量	实际行为	执行结果
-1	拒绝	ERROR 3819
0	允许	Query OK
100	允许	Query OK
101	拒绝	ERROR 3819

13. 建立供应商编号唯一索引

```
CREATE UNIQUE INDEX uq_supplier_no ON Supplier (SupplierNO);
```

执行结果与分析：创建 uq_supplier_no 后，SHOW INDEX 同时列出 PRIMARY 和该唯一索引，Non_unique 均为 0。供应商编号已经是主码，该索引在实际设计中属于重复索引，此处仅为完成创建与删除索引的练习。

14. 建立类别编号升序与类别名降序索引

```
CREATE UNIQUE INDEX uq_category_no_name
  ON Category (CategoryNO ASC, CategoryName DESC);
```

执行结果与分析：原题要求 Category 的“商品名”降序，但该表没有 GoodsName，只有商品种类名 CategoryName，因此使用 CategoryName。元数据中两个索引列的 COLLATION 分别为 A 和 D，NON_UNIQUE 为 0，验证了升序、降序和唯一性定义。

15. 删除供应商编号唯一索引

```
DROP INDEX uq_supplier_no ON Supplier;
```

执行结果与分析：DROP INDEX 执行成功。SHOW INDEX 中只保留 Supplier 的 PRIMARY，uq_supplier_no 已删除。主码仍保证供应商编号唯一，Goods 对 Supplier 的外键也仍然存在。

样本数据及完整性验证

15 项任务均执行成功。除结构查询外，共完成 17 项实际行为验证。验证使用指导书样本，Category、Supplier、Student、Goods、SaleBill 分别导入 3、5、5、4、6 行。

Student 的出生年份已按第 8 项删除，因此验证只写入剩余属性。样本仅用于验证数据定义任务，未执行实验二至四的操作。

验证项目	实际结果
删除唯一约束前重复非空微信号	ERROR 1062，第二条插入被拒绝
删除唯一约束后重复微信号	更新成功，同微信号记录数为 2
Number 大于 0 时写入 0	ERROR 3819
范围约束下写入 -1、101	均返回 ERROR 3819
范围约束下写入 0、100	均成功
类别主码重复、销售复合主码重复	均返回 ERROR 1062
四个外键引用不存在的父记录	均返回 ERROR 1452
删除仍被商品引用的供应商	ERROR 1451
将销售数量改为 NULL	ERROR 1048
进价写入 2.56	保存为 2.6，符合一位小数定义
五表样本数量及全部验证后状态	导入数量符合预期，回滚后共 0 行

失败用例中的错误是约束生效的预期结果。成功用例显式执行 ROLLBACK；失败用例结束连接后，未提交事务自动回滚。所有验证结束后再次查询，五张表的数据行数均为 0。实验结果文件保留了执行的 SQL、标准输出和 MySQL 错误码，可以复核上述结论。

七、实验思考

1. MySQL 提供的系统数据类型有哪些？

常用类型包括整数类型 TINYINT、SMALLINT、MEDIUMINT、INT、BIGINT，精确数值类型 DECIMAL，浮点类型 FLOAT、DOUBLE；日期与时间类型 DATE、TIME、

DATETIME、TIMESTAMP、YEAR；字符串和二进制类型 CHAR、VARCHAR、TEXT、BINARY、VARBINARY、BLOB，以及 BIT、ENUM、SET、JSON 和空间类型等。本实验使用 VARCHAR 保存编号及中文名称，用 DATE 保存日期，用 DECIMAL 保存需要固定小数位的价格。

2. 什么是主码和外码？它们的作用是什么？

主码是关系中用于唯一标识一条记录的候选码，可以由一个或多个属性组成，必须唯一且非空。外码是引用其它关系候选码的属性或属性组合；本实验的外码均引用父表主码。主码保证实体完整性，外码保证参照完整性，防止销售记录引用不存在的商品或学生。

3. 定义表结构描述数据模型的哪个要素？对主属性有什么要求？

定义表结构主要描述数据模型的数据结构，同时通过约束表达完整性要求。主属性是属于任一候选码的属性；选择主码后，构成主码的属性必须满足非空要求，整体必须唯一。销售表的 GoodsNO 与 SNO 单独可以重复，但二者的组合必须唯一。

4. 修改表结构时能否修改属性参数？新增列能否定义为非空？

可以通过 ALTER TABLE MODIFY COLUMN 修改类型、长度、精度和可空性等参数，但已有数据必须满足新定义，否则可能报错或发生转换、舍入。新增列可以定义为 NOT NULL；表中已有记录时，应提供符合业务的默认值，或先增加可空列、补齐数据，再改为非空。还需检查已有外键、索引和检查约束是否依赖该列，避免不兼容修改。

5. 能否在非主属性列上建立索引？恰当方式是什么？

可以。应根据筛选、连接和排序方式选择普通索引或复合索引，只有在业务要求唯一时才使用唯一索引。索引会占用空间并增加写入维护成本，应避免无用及重复索引。本实验供应商编号已是主码，再建立同列唯一索引只用于演示，删除后主码的唯一性仍然存在。

八、实验总结

本次完成了五张关系表的定义、属性列调整、检查约束替换以及索引创建和删除。实际验证表明，检查约束、唯一约束及外键不仅是结构说明，也会在插入、更新和删除时阻止不符合规则的数据。

实验中需要注意父表与子表的定义顺序、电话等属性的真实含义，以及指导书表结构与任务文字的差异。使用命名约束和保存执行日志，便于准确修改结构并复核结果。数据定义语句会产生隐式提交，不能依赖普通事务回滚撤销本次表结构修改；样本验证则使用 InnoDB 事务，避免影响实验完成后的数据状态。

实验二　数据更新

一、学时

4 学时。

二、实验类型

设计。

三、实验目的

1. 巩固数据库的基础知识。
2. 熟悉和掌握数据插入、修改、删除的方法。
3. 加深理解表定义对数据更新的作用。
4. 理解数据库完整性规则对更新操作的约束。

四、实验准备

1. 熟悉 SQL Server 的工作环境。
2. 复习对表中数据的插入、修改和删除的 SQL 语言命令。

五、实验方法及步骤

1. 登录企业管理器 / 查询分析器。
2. 连接已建立的数据库 supermarket。
3. 在当前数据库上完成数据插入、修改和删除任务。

六、实验结果

1. 将五张表的样本数据导入数据库

```
-- 原始五表样本；销售商品号统一到 Goods 主码。
INSERT INTO Category (CategoryNO, CategoryName, Descriptions) VALUES
 ('CN001', '咖啡', '速溶咖啡，袋装咖啡，咖啡粉'),
 ('CN002', '洗发水', '袋装、瓶装洗发水'),
 ('CN005', '床上用品', '被套，枕套，床单');
INSERT INTO Supplier (SupplierNO, SupplierName, Address, Number) VALUES
 ('sup001', '卡夫食品', '广东佛山', '12348768900'),
 ('sup002', '久润食品', '广东东莞', '13248768901'),
 ('sup003', '飞鹤食品', '重庆解放碑', '12648768901'),
 ('sup004', '南山日化', '重庆南坪', '11648768903'),
 ('sup005', '缙云日化', '重庆北碚', '19648768903');
INSERT INTO Student (SNO, SName, BirthYear, Gender, College, Major, WeiXin) VALUES
 ('S01', '李明', 1999, '男', 'CS', 'IT', 'wx001'),
 ('S02', '徐好', 1998, '女', 'CS', 'MIS', 'wx002'),
```

```
 ('S03', '伍民', 1996, '男', 'CS', 'MIS', 'wx003'),
 ('S04', '闵红', 1997, '女', 'ACC', 'AC', 'wx004'),
 ('S05', '张小红', 1997, '女', 'ACC', 'AC', 'wx005');
INSERT INTO Goods
 (GoodsNO, SupplierNO, CategoryNO, GoodsName,
  InPrice, SalePrice, Number, ProductTime) VALUES
 ('GN0001', 'sup001', 'CN001', '优乐美奶茶', 2.5, 3.5, 100, '2019-12-20'),
 ('GN0002', 'sup002', 'CN001', '雀巢咖啡', 4, 5.8, 50, '2017-06-08'),
 ('GN0005', 'sup003', 'CN002', '飘柔洗发水', 15, 19.8, 65, '2019-05-29'),
 ('GN0007', 'sup005', 'CN005', '小绵羊被套', 120, 150, 28, '2019-11-22');
INSERT INTO SaleBill (GoodsNO, SNO, HappenTime, Number) VALUES
 ('GN0001', 'S01', '2020-06-09', 3),
 ('GN0001', 'S02', '2020-05-03', 1),
 ('GN0001', 'S03', '2020-04-07', 1),
 ('GN0002', 'S02', '2018-05-08', 2),
 ('GN0002', 'S05', '2019-06-26', 2),
 ('GN0005', 'S05', '2020-06-01', 1);

SELECT 'Category' AS table_name,COUNT(*) AS rows_count FROM Category
UNION ALL SELECT 'Supplier',COUNT(*) FROM Supplier
UNION ALL SELECT 'Student',COUNT(*) FROM Student
UNION ALL SELECT 'Goods',COUNT(*) FROM Goods
UNION ALL SELECT 'SaleBill',COUNT(*) FROM SaleBill;
```

table_name	rows_count
Category	3
Supplier	5
Student	5
Goods	4
SaleBill	6

结果分析：按父表、子表的依赖顺序导入。各表行数依次为 3、5、5、4、6。

2. 将雀巢咖啡改名为雀巢咖啡条装

操作 1：

```
UPDATE Goods SET GoodsName='雀巢咖啡条装' WHERE GoodsName='雀巢咖啡';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT GoodsNO,GoodsName FROM Goods WHERE GoodsNO='GN0002';
```

GoodsNO	GoodsName
GN0002	雀巢咖啡条装

结果分析：修改了 GN0002 的商品名，影响 1 行。

3. 将所有商品的售价提升百分之五

操作 1：

```
UPDATE Goods SET SalePrice=ROUND(SalePrice*1.05,2);
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：4。

操作 2：

```
SELECT GoodsNO,SalePrice FROM Goods ORDER BY GoodsNO;
```

GoodsNO	SalePrice
GN0001	3.68
GN0002	6.09
GN0005	20.79
GN0007	157.50

结果分析：更新四件商品，DECIMAL(10,2) 保存两位小数。

4. 将类别号 CN005 改为 CN004

操作 1：

```
UPDATE Category SET CategoryNO='CN004' WHERE CategoryNO='CN005';
```

实际返回 ERROR 1451，符合预期的完整性限制。

操作 2：

```
SELECT CategoryNO FROM Category WHERE CategoryNO IN ('CN004','CN005');
```

CategoryNO
CN005

结果分析：直接修改被 GN0007 引用的父表主码，返回 ERROR 1451。外键采用默认 RESTRICT，不允许留下无效引用；类别号保持 CN005。若业务确需变更，可设计 ON UPDATE CASCADE，或在事务内新增父记录、迁移子表引用后删除旧父记录。

5. 将 2020 年销售总数量小于三的商品售价打八折

操作 1：

```
UPDATE Goods AS g
JOIN (SELECT GoodsNO FROM SaleBill
      WHERE HappenTime >= '2020-01-01' AND HappenTime < '2021-01-01'
      GROUP BY GoodsNO HAVING SUM(Number)<3) AS t
  ON t.GoodsNO=g.GoodsNO
SET g.SalePrice=ROUND(g.SalePrice*0.8,2);
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT GoodsNO,SalePrice FROM Goods ORDER BY GoodsNO;
```

GoodsNO	SalePrice
GN0001	3.68
GN0002	6.09
GN0005	16.63
GN0007	157.50

结果分析：“销售数量”按该商品 2020 年合计销量理解，只包含该年确有销售记录的商品。GN0001 的年销量为 5，不打折；GN0005 为 1，按前题提升后的售价计算，结果为 16.63。

6. 将久润食品供应商品的库存数量清零

操作 1：

```
UPDATE Goods AS g JOIN Supplier AS s
  ON s.SupplierNO=g.SupplierNO
SET g.Number=0 WHERE s.SupplierName='久润食品';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT GoodsNO,Number FROM Goods WHERE GoodsNO='GN0002';
```

GoodsNO	Number
GN0002	0

结果分析：Number 在 Goods 中表示库存，GN0002 的库存改为 0，销售记录数量保持不变。

7. 删除购买优乐美奶茶且数量低于三的销售记录

操作 1：

```
DELETE b FROM SaleBill AS b JOIN Goods AS g
  ON g.GoodsNO=b.GoodsNO
WHERE g.GoodsName='优乐美奶茶' AND b.Number<3;
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：2。

操作 2：

```
SELECT SNO,Number FROM SaleBill WHERE GoodsNO='GN0001';
```

SNO	Number
S01	3

结果分析：删除 S02、S03 各购买 1 件的记录，S01 购买 3 件的记录保留。

8. 删除 2018 年以前生产的商品并处理外键引用

操作 1：

```
DELETE FROM Goods WHERE ProductTime<'2018-01-01';
```

实际返回 ERROR 1451，符合预期的完整性限制。

操作 2：

```
START TRANSACTION;
DELETE b FROM SaleBill AS b JOIN Goods AS g ON g.GoodsNO=b.GoodsNO
WHERE g.ProductTime<'2018-01-01';
DELETE FROM Goods WHERE ProductTime<'2018-01-01';
COMMIT;
```

语句执行成功。

操作 3：

```
SELECT GoodsNO FROM Goods ORDER BY GoodsNO;
```

GoodsNO
GN0001
GN0005
GN0007

结果分析：GN0002 尚有两条销售记录，直接删除返回 ERROR 1451。正确处理是先删除对应销售记录，再删除商品；两个动作放在同一事务中，商品与销售记录一起提交。

9. 删除卡夫食品供应商品的销售记录

```
DELETE b FROM SaleBill AS b
JOIN Goods AS g ON g.GoodsNO=b.GoodsNO
JOIN Supplier AS s ON s.SupplierNO=g.SupplierNO
WHERE s.SupplierName='卡夫食品';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

结果分析：前题操作后，卡夫食品对应剩余 GN0001/S01 一条销售记录，删除成功。

10. 删除 ACC 学院学生的销售记录

```
DELETE b FROM SaleBill AS b JOIN Student AS s
ON s.SNO=b.SNO WHERE s.College='ACC';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

结果分析：删除张小红购买 GN0005 的剩余记录，影响 1 行；不删除学生本身。

11. 按引用顺序删除所有销售记录和商品

操作 1：

```
START TRANSACTION;
DELETE FROM SaleBill;
DELETE FROM Goods;
COMMIT;
```

语句执行成功。

操作 2：

```
SELECT (SELECT COUNT(*) FROM SaleBill) AS sales,
       (SELECT COUNT(*) FROM Goods) AS goods;
```

sales	goods
0	0

结果分析：先删除 SaleBill，再删除 Goods。此时销售表已空，商品表剩余 3 行，事务提交后两表均空。

12. 删除 1997 年以前出生的学生

操作 1：

```
DELETE FROM Student WHERE BirthYear<1997;
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT SNO,SName,BirthYear FROM Student ORDER BY SNO;
```

SNO	SName	BirthYear
S01	李明	1999
S02	徐好	1998
S04	闵红	1997
S05	张小红	1997

结果分析：1996 年出生的伍民被删除，剩余 4 名学生。此时销售表已空，不会发生外键冲突。

补充验证与最终状态

除以上任务，还保存了 3 项补充验证记录。临时补充数据在事务中回滚，没有写入原始样本作为题目答案。

验证项目	实际结果
事务内新增类别并迁移引用	通过
类别迁移验证后回滚	通过
旧商品的销售记录一并删除	通过

最终表行数：

关系表	行数
Category	3
Supplier	5
Student	4
Goods	0
SaleBill	0

原始 SQL、逐题执行日志、结构化汇总与最终数据库导出已保存，可根据记录复核结果。

七、实验思考

1. 使用 SQL 为表插入数据时应注意什么？

应明确列名、列顺序、类型、长度和非空要求，正确表示字符串与日期，并检查主码、唯一约束、检查约束和外键。存在引用关系时先插入父表再插入子表。本实验恢复出生年份；视图插入题未提供性别，允许该列为 NULL。

2. 一次 UPDATE 可以更新几个表？

MySQL 支持单表 UPDATE，也支持多表 UPDATE。多表更新仍受外键和检查约束限制，不能只依据书写顺序假定数据库按期望的依赖顺序处理。复杂关联修改可以拆成事务内的明确步骤。

3. 涉及多表更新时应注意什么？

先核对引用关系及筛选条件，用事务保证相关 DML 一起成功或回滚。删除父记录前先处理子记录；修改父码可采用级联更新，或先建立新父记录再迁移子表引用。不得为了让语句通过而关闭外键检查。

4. 定义和未定义主码时更新 Goods 有什么不同？

有主码时数据库拒绝重复或为空的商品编号；没有主码时可能产生无法唯一标识的记录，导致条件更新误改多条记录。删除或修改普通属性仍须注意 WHERE 条件。

5. 定义和未定义外码时更新 Goods 有什么不同？

有外键时商品必须引用存在的供应商和类别，删除或修改被引用的父记录也受限制。没有外键时数据库不自动阻止悬空引用，完整性只能依靠应用逻辑或人工检查。

6. 如果 Goods 与 Category 之间没有外键，更新会出现什么情况？

把 CN005 改成 CN004 后，Goods 中仍可能留下 CN005，连接查询无法取得商品类别名称。本次外键实际拒绝该更新；补充验证在事务中迁移引用可以成功，验证后已回滚。

八、实验总结

本次完成五表数据导入以及更新、删除任务。外键阻止了直接修改被引用的类别码和删除仍有销售记录的商品；通过先处理子表引用，可以在保留完整性约束的前提下完成目标操作。各题按顺序执行，因此后续影响行数依据前题完成后的状态计算。

实验三　数据查询

一、学时

4 学时。

二、实验类型

设计。

三、实验目的

1. 巩固数据库基础知识，理解查询的概念和方法。
2. 掌握数据表查询及 SQL 命令的使用。
3. 熟悉连接查询、分组统计、子查询和派生表。
4. 能够根据题意选择合适的查询方法并解释结果。

四、实验准备

1. 熟悉 SQL Server 的工作环境。
2. 复习有关表的数据查询的 SQL 语言命令和方法。

五、实验方法及步骤

1. 登录企业管理器 / 查询分析器。
2. 连接已建立的数据库。
3. 在当前数据库上完成数据查询任务。

六、实验结果

1. 查询全部商品信息

```
SELECT * FROM Goods ORDER BY GoodsNO;
```

GoodsNO	SupplierNO	CategoryNO	GoodsName	InPrice
GN0001	sup001	CN001	优乐美奶茶	2.5
GN0002	sup002	CN001	雀巢咖啡	4.0
GN0005	sup003	CN002	飘柔洗发水	15.0
GN0007	sup005	CN005	小绵羊被套	120.0

GoodsNO	SalePrice	Number	ProductTime
GN0001	3.50	100	2019-12-20
GN0002	5.80	50	2017-06-08
GN0005	19.80	65	2019-05-29
GN0007	150.00	28	2019-11-22

结果分析：返回原始样本中的 4 件商品。

2. 查询类别 cn002 的商品

```
SELECT * FROM Goods WHERE CategoryNO='cn002' ORDER BY GoodsNO;
```

GoodsNO	SupplierNO	CategoryNO	GoodsName	InPrice
GN0005	sup003	CN002	飘柔洗发水	15.0

GoodsNO	SalePrice	Number	ProductTime
GN0005	19.80	65	2019-05-29

结果分析：排序规则不区分英文字母大小写，返回飘柔洗发水。

3. 查询类别 cn001 或 cn002 的商品

```
SELECT * FROM Goods WHERE CategoryNO IN ('cn001','cn002')
ORDER BY GoodsNO;
```

GoodsNO	SupplierNO	CategoryNO	GoodsName	InPrice
GN0001	sup001	CN001	优乐美奶茶	2.5
GN0002	sup002	CN001	雀巢咖啡	4.0
GN0005	sup003	CN002	飘柔洗发水	15.0

GoodsNO	SalePrice	Number	ProductTime
GN0001	3.50	100	2019-12-20
GN0002	5.80	50	2017-06-08
GN0005	19.80	65	2019-05-29

结果分析：IN 表达两种类别之一，返回 3 件商品。

4. 查询 cn005 且进价大于二十的商品

```
SELECT * FROM Goods WHERE CategoryNO='cn005' AND InPrice>20;
```

GoodsNO	SupplierNO	CategoryNO	GoodsName	InPrice
GN0007	sup005	CN005	小绵羊被套	120.0

GoodsNO	SalePrice	Number	ProductTime
GN0007	150.00	28	2019-11-22

结果分析：两个条件用 AND 连接，返回小绵羊被套。

5. 查询库存数量小于十的商品

```
SELECT GoodsNO,SupplierNO,GoodsName,Number
FROM Goods WHERE Number<10 ORDER BY GoodsNO;
```

语句执行成功。

结果分析：原始库存最小值为 28，因此返回空集。

6. 查询有购买记录的学生学号

```
SELECT DISTINCT SNO FROM SaleBill ORDER BY SNO;
```

SNO
S01
S02
S03
S05

结果分析：DISTINCT 去除同一学生多次购买产生的重复学号，返回 4 人。

7. 查询 IT 专业学生学号姓名和年龄

```
SELECT SNO,SName,YEAR(CURRENT_DATE())-BirthYear AS Age
FROM Student WHERE Major='IT' ORDER BY SNO;
```

SNO	SName	Age
S01	李明	27

结果分析：使用本次执行年份减出生年份得到年历年龄。只有李明属于 IT；当前年龄值见实际结果。

8. 查询年龄二十二至二十四的学生

```
SELECT SNO,SName,College,Major,YEAR(CURRENT_DATE())-BirthYear AS Age
FROM Student WHERE YEAR(CURRENT_DATE())-BirthYear BETWEEN 22 AND 24
ORDER BY SNO;
```

语句执行成功。

结果分析：本次查询年份为 2026，样本学生年历年龄为 27 ~ 30，返回空集。没有把系统时间改成教学年份。

9. 查询姓张的学生

```
SELECT SNO,SName,College FROM Student WHERE SName LIKE '张%'
ORDER BY SNO;
```

SNO	SName	College
S05	张小红	ACC

结果分析：LIKE 的前缀匹配返回张小红。

10. 查询库存二十以上商品并按数量降序

```
SELECT * FROM Goods WHERE Number>=20 ORDER BY Number DESC,GoodsNO;
```

GoodsNO	SupplierNO	CategoryNO	GoodsName	InPrice
GN0001	sup001	CN001	优乐美奶茶	2.5
GN0005	sup003	CN002	飘柔洗发水	15.0
GN0002	sup002	CN001	雀巢咖啡	4.0
GN0007	sup005	CN005	小绵羊被套	120.0

GoodsNO	SalePrice	Number	ProductTime
GN0001	3.50	100	2019-12-20
GN0005	19.80	65	2019-05-29
GN0002	5.80	50	2017-06-08
GN0007	150.00	28	2019-11-22

结果分析：四件商品均满足条件，数量依次为 100、65、50、28。

11. 查询库存十以内商品并按类别和库存排序

```
SELECT GoodsNO,GoodsName,CategoryNO,Number FROM Goods
WHERE Number<=10 ORDER BY CategoryNO ASC,Number DESC,GoodsNO;
```

语句执行成功。

结果分析：按包含 10 理解“十以内”，当前样本没有满足条件的商品。

12. 查询库存数量排名前三的商品

```
SELECT * FROM Goods ORDER BY Number DESC,GoodsNO LIMIT 3;
```

GoodsNO	SupplierNO	CategoryNO	GoodsName	InPrice
GN0001	sup001	CN001	优乐美奶茶	2.5
GN0005	sup003	CN002	飘柔洗发水	15.0
GN0002	sup002	CN001	雀巢咖啡	4.0

GoodsNO	SalePrice	Number	ProductTime
GN0001	3.50	100	2019-12-20
GN0005	19.80	65	2019-05-29
GN0002	5.80	50	2017-06-08

结果分析：使用 LIMIT 3，数量相同时以商品编码确定输出顺序。

13. 查询 cn001 库存最大的商品

```
SELECT GoodsNO,GoodsName,Number FROM Goods
WHERE CategoryNO='cn001' AND Number=(
  SELECT MAX(Number) FROM Goods WHERE CategoryNO='cn001')
ORDER BY GoodsNO;
```

GoodsNO	GoodsName	Number
GN0001	优乐美奶茶	100

结果分析：使用最大值子查询，若有并列最大值会全部返回。

14. 统计 MIS 专业学生人数

```
SELECT COUNT(*) AS StudentCount FROM Student WHERE Major='MIS';
```

StudentCount
2

结果分析：MIS 专业有徐好、伍民两人。

15. 按学院统计二十岁以上人数

```
SELECT College,COUNT(*) AS StudentCount FROM Student
WHERE YEAR(CURRENT_DATE())-BirthYear>20
GROUP BY College ORDER BY College;
```

College	StudentCount
ACC	2
CS	3

结果分析：先按年龄过滤，再按学院分组，ACC 为 2 人、CS 为 3 人。

16. 统计 2018 年各商品销量

```
SELECT GoodsNO,SUM(Number) AS SaleTotal FROM SaleBill
WHERE HappenTime>='2018-01-01' AND HappenTime<'2019-01-01'
GROUP BY GoodsNO ORDER BY GoodsNO;
```

GoodsNO	SaleTotal
GN0002	2

结果分析：按销售日期筛选，2018 年只有 GN0002 销售 2 件。

17. 查询学生人数在三人以上的学院

```
SELECT College,COUNT(*) AS StudentCount FROM Student
GROUP BY College HAVING COUNT(*)>=3 ORDER BY College;
```

College	StudentCount
CS	3

结果分析：“三人以上”按包含 3 理解，CS 满足条件；HAVING 用于过滤分组结果。

18. 查询 2019 年购买总数超过五的学生

```
SELECT SNO,SUM(Number) AS BuyTotal FROM SaleBill
WHERE HappenTime>='2019-01-01' AND HappenTime<'2020-01-01'
GROUP BY SNO HAVING SUM(Number)>5 ORDER BY SNO;
```

语句执行成功。

结果分析：该年只有 S05 购买 2 件，因此返回空集。

19. 查询咖啡类商品平均售价

```
SELECT CAST(AVG(g.SalePrice) AS DECIMAL(10,2)) AS AvgPrice
FROM Goods AS g JOIN Category AS c ON c.CategoryNO=g.CategoryNO
WHERE c.CategoryName='咖啡';
```

AvgPrice
4.65

结果分析：按 CategoryName 判断类别；奶茶归在咖啡类，按该分类计算，平均价为 4.65。

20. 查询与张小红同学院的学生

```
SELECT SNO,SName,College FROM Student WHERE College IN
(SELECT College FROM Student WHERE SName='张小红') ORDER BY SNO;
```

SNO	SName	College
S04	闵红	ACC
S05	张小红	ACC

结果分析：题目没有排除本人，因此返回闵红和张小红。

21. 查询全部销售情况并按日期降序

```
SELECT g.GoodsNO,g.GoodsName,s.SName,b.HappenTime,b.Number
FROM SaleBill AS b JOIN Goods AS g ON g.GoodsNO=b.GoodsNO
JOIN Student AS s ON s.SNO=b.SNO
ORDER BY b.HappenTime DESC,g.GoodsNO,s.SNO;
```

GoodsNO	GoodsName	SName	HappenTime	Number
GN0001	优乐美奶茶	李明	2020-06-09	3
GN0005	飘柔洗发水	张小红	2020-06-01	1
GN0001	优乐美奶茶	徐好	2020-05-03	1
GN0001	优乐美奶茶	伍民	2020-04-07	1
GN0002	雀巢咖啡	张小红	2019-06-26	2
GN0002	雀巢咖啡	徐好	2018-05-08	2

结果分析：销售记录连接商品及学生，按日期降序返回 6 条明细。

22. 查询购买商品的学生

```
SELECT s.SNO,s.SName,s.College FROM Student AS s
WHERE EXISTS (SELECT 1 FROM SaleBill AS b WHERE b.SNO=s.SNO)
ORDER BY s.SNO;
```

SNO	SName	College
S01	李明	CS
S02	徐好	CS
S03	伍民	CS
S05	张小红	ACC

结果分析：EXISTS 避免重复返回同一学生。

23. 查询购买咖啡类商品的学生

```
SELECT DISTINCT s.SNO,s.SName,s.College FROM Student AS s
JOIN SaleBill AS b ON b.SNO=s.SNO
JOIN Goods AS g ON g.GoodsNO=b.GoodsNO
JOIN Category AS c ON c.CategoryNO=g.CategoryNO
WHERE c.CategoryName='咖啡' ORDER BY s.SNO;
```

SNO	SName	College
S01	李明	CS
S02	徐好	CS
S03	伍民	CS
S05	张小红	ACC

结果分析：连接四表后 DISTINCT 去重，返回 4 人。

24. 用连接和子查询查询未出售商品

操作 1：

```
SELECT g.GoodsNO,g.GoodsName,c.CategoryName,g.SalePrice,g.Number
FROM Goods AS g JOIN Category AS c ON c.CategoryNO=g.CategoryNO
LEFT JOIN SaleBill AS b ON b.GoodsNO=g.GoodsNO
WHERE b.GoodsNO IS NULL ORDER BY g.GoodsNO;
```

GoodsNO	GoodsName	CategoryName	SalePrice	Number
GN0007	小绵羊被套	床上用品	150.00	28

操作 2：

```
SELECT g.GoodsNO,g.GoodsName,c.CategoryName,g.SalePrice,g.Number
FROM Goods AS g JOIN Category AS c ON c.CategoryNO=g.CategoryNO
WHERE NOT EXISTS (SELECT 1 FROM SaleBill AS b
                  WHERE b.GoodsNO=g.GoodsNO) ORDER BY g.GoodsNO;
```

GoodsNO	GoodsName	CategoryName	SalePrice	Number
GN0007	小绵羊被套	床上用品	150.00	28

结果分析：分别使用 LEFT JOIN 判空和 NOT EXISTS，两个结果相同，均为 GN0007。

25. 查询买过 gn001 或 gn002 的学生

```
SELECT DISTINCT s.SNO,s.SName,s.College FROM Student AS s
JOIN SaleBill AS b ON b.SNO=s.SNO
WHERE b.GoodsNO IN ('GN0001','GN0002') ORDER BY s.SNO;
```

SNO	SName	College
S01	李明	CS
S02	徐好	CS
S03	伍民	CS
S05	张小红	ACC

结果分析：商品编号使用 GN0001、GN0002，OR 表达任意一种。

26. 查询同时买过 gn001 和 gn002 的学生

```
SELECT s.SNO,s.SName,s.College FROM Student AS s
JOIN SaleBill AS b ON b.SNO=s.SNO
WHERE b.GoodsNO IN ('GN0001','GN0002')
GROUP BY s.SNO,s.SName,s.College
HAVING COUNT(DISTINCT b.GoodsNO)=2 ORDER BY s.SNO;
```

SNO	SName	College
S02	徐好	CS

结果分析：先限定两种商品，再要求去重商品数为 2，只有徐好同时购买。

27. 用相关子查询和派生表查询各学院年龄最大者

操作 1：

```
SELECT s.SNO,s.SName,s.College FROM Student AS s
WHERE s.BirthYear=(SELECT MIN(t.BirthYear) FROM Student AS t
                   WHERE t.College=s.College) ORDER BY s.SNO;
```

SNO	SName	College
S03	伍民	CS
S04	闵红	ACC
S05	张小红	ACC

操作 2：

```
SELECT s.SNO,s.SName,s.College FROM Student AS s
JOIN (SELECT College,MIN(BirthYear) AS MinYear FROM Student
      GROUP BY College) AS t
ON t.College=s.College AND t.MinYear=s.BirthYear ORDER BY s.SNO;
```

SNO	SName	College
S03	伍民	CS
S04	闵红	ACC
S05	张小红	ACC

结果分析：出生年份越小年龄越大；ACC 有两名并列，必须一起返回。两种实现结果一致。

28. 统计每个商品的销售总量

```
SELECT g.GoodsNO,g.GoodsName,COALESCE(SUM(b.Number),0) AS SaleTotal
FROM Goods AS g LEFT JOIN SaleBill AS b ON b.GoodsNO=g.GoodsNO
GROUP BY g.GoodsNO,g.GoodsName ORDER BY g.GoodsNO;
```

GoodsNO	GoodsName	SaleTotal
GN0001	优乐美奶茶	5
GN0002	雀巢咖啡	4
GN0005	飘柔洗发水	1
GN0007	小绵羊被套	0

结果分析：LEFT JOIN 保留未出售商品，COALESCE 将其销售总量记为 0。

29. 查询 2019 年购买总数超过五的学生姓名和学院

```
SELECT s.SName,s.College,SUM(b.Number) AS BuyTotal
FROM Student AS s JOIN SaleBill AS b ON b.SNO=s.SNO
WHERE b.HappenTime>='2019-01-01' AND b.HappenTime<'2020-01-01'
GROUP BY s.SNO,s.SName,s.College HAVING SUM(b.Number)>5
ORDER BY s.SNO;
```

语句执行成功。

结果分析：与第 18 项同一条件，连接 Student 增加姓名和学院，本样本返回空集。

30. 查询供应总数超过五十的供应商

```
SELECT s.SupplierNO,s.SupplierName,s.Address,SUM(g.Number) AS SupplyTotal
FROM Supplier AS s JOIN Goods AS g ON g.SupplierNO=s.SupplierNO
GROUP BY s.SupplierNO,s.SupplierName,s.Address
HAVING SUM(g.Number)>50 ORDER BY SupplyTotal DESC,s.SupplierNO;
```

SupplierNO	SupplierName	Address	SupplyTotal
sup001	卡夫食品	广东佛山	100
sup003	飞鹤食品	重庆解放碑	65

结果分析：“供应商品总数”按 Goods.Number 的库存数量求和，不按商品种类数计算。返回卡夫食品 100 件、飞鹤食品 65 件。

补充验证与最终状态

除以上任务，还保存了 7 项补充验证记录。临时补充数据在事务中回滚，没有写入原始样本作为题目答案。

验证项目	实际结果
补充低库存商品后返回两行	通过
类别升序、库存降序排序	通过
补充 23 岁学生后命中年龄区间	通过
补充 2019 年购买 6 件后满足 HAVING	通过
对应姓名、学院连接结果一致	通过
最大库存并列时返回全部商品	通过
补充验证后五表行数恢复	通过

最终表行数：

关系表	行数
Category	3
Supplier	5
Student	5
Goods	4
SaleBill	6

原始 SQL、逐题执行日志、结构化汇总与最终数据库导出已保存，可根据记录复核结果。

七、实验思考

1. 如何区分 WHERE 和 HAVING？

WHERE 在分组前过滤明细，HAVING 在分组后过滤聚合结果。统计某年度销量时，应先限制销售日期，再按商品或学生分组，最后按合计数量筛选。

2. 为什么有些查询没有记录？

空集取决于样本和查询条件，不等于 SQL 执行失败。本次数据没有低库存商品、22 ~ 24 岁学生或 2019 年购买超过 5 件的学生，已用事务内补充样本验证这些条件会返回相应记录，并验证后回滚。

3. 如何正确处理重复、并列和未匹配记录？

学生多次购买时用 DISTINCT 或 EXISTS 去重；最大年龄或最大库存的并列值用等值子查询保留全部；统计所有商品销量时用 LEFT JOIN 和 COALESCE 保留未出售商品的零销量。

八、实验总结

本次完成 30 项查询，并验证了两种未出售商品查询和两种学院最大年龄查询结果一致。查询除了语法正确，还应明确计数与求和、或与且、筛选与分组、并列最大值以及空集的含义。年龄按实际执行年份计算，没有为了匹配教材年龄范围而改写日期。

实验四　视图操作

一、学时

2 学时。

二、实验类型

设计。

三、实验目的

1. 掌握视图定义和查询操作。
2. 理解视图更新与基本表更新的联系。
3. 掌握可更新视图的使用条件。
4. 认识视图在简化查询和限制数据访问方面的作用。

四、实验准备

1. 熟悉 SQL Server 的工作环境。
2. 复习有关视图操作的 SQL 语言命令。

五、实验方法及步骤

1. 登录企业管理器 / 查询分析器。
2. 连接已建立的数据库 supermarket。
3. 在当前数据库上完成视图的定义、查询和更新操作。

六、实验结果

1. 定义 MIS 专业学生视图 V_MIS 并查询

操作 1：

```
CREATE VIEW V_MIS AS SELECT * FROM Student WHERE Major='MIS';
```

语句执行成功。

操作 2：

```
SELECT SNO,SName,Major FROM V_MIS ORDER BY SNO;
```

SNO	SName	Major
S02	徐好	MIS
S03	伍民	MIS

结果分析：采用简单单表视图，不加 WITH CHECK OPTION，便于观察后续插入 IT 学生的行为。初始可见徐好、伍民。

2. 定义学生购买商品信息视图 V_SSG 并查询

操作 1：

```
CREATE VIEW V_SSG AS
SELECT s.SNO,s.SName,g.GoodsName,g.SalePrice,b.Number
FROM Student AS s JOIN SaleBill AS b ON b.SNO=s.SNO
JOIN Goods AS g ON g.GoodsNO=b.GoodsNO;
```

语句执行成功。

操作 2：

```
SELECT * FROM V_SSG ORDER BY SNO,GoodsName;
```

SNO	SName	GoodsName	SalePrice	Number
S01	李明	优乐美奶茶	3.50	3
S02	徐好	优乐美奶茶	3.50	1
S02	徐好	雀巢咖啡	5.80	2
S03	伍民	优乐美奶茶	3.50	1
S05	张小红	雀巢咖啡	5.80	2
S05	张小红	飘柔洗发水	19.80	1

结果分析：视图将学生、销售和商品关联，返回 6 条记录；每列名称唯一。

3. 定义学生年龄视图 V_AGE 并查询

操作 1：

```
CREATE VIEW V_AGE AS SELECT SNO,SName,College,Major,
YEAR(CURRENT_DATE())-BirthYear AS Age FROM Student;
```

语句执行成功。

操作 2：

```
SELECT SNO,SName,Age FROM V_AGE ORDER BY SNO;
```

SNO	SName	Age
S01	李明	27
S02	徐好	28
S03	伍民	30
S04	闵红	29
S05	张小红	29

结果分析：年历年龄由当前年份减出生年份计算，当前年份为 2026。

4. 定义学院人数及平均年龄视图 V_COM 并查询

操作 1：

```
CREATE VIEW V_COM AS SELECT College,COUNT(*) AS StudentCount,
CAST(AVG(YEAR(CURRENT_DATE())-BirthYear) AS DECIMAL(6,2)) AS AvgAge
FROM Student GROUP BY College;
```

语句执行成功。

操作 2：

```
SELECT * FROM V_COM ORDER BY College;
```

College	StudentCount	AvgAge
ACC	2	29.00
CS	3	28.33

结果分析：包含 GROUP BY 和聚合函数，属于不可更新视图。

5. 定义学生购买种类及消费总额视图 V_S_CONSUME

操作 1：

```
CREATE VIEW V_S_CONSUME AS
SELECT s.SNO,s.SName,s.College,COUNT(DISTINCT b.GoodsNO) AS GoodsKinds,
CAST(COALESCE(SUM(b.Number*g.SalePrice),0) AS DECIMAL(12,2)) AS CostTotal
FROM Student AS s LEFT JOIN SaleBill AS b ON b.SNO=s.SNO
LEFT JOIN Goods AS g ON g.GoodsNO=b.GoodsNO
GROUP BY s.SNO,s.SName,s.College;
```

语句执行成功。

操作 2：

```
SELECT * FROM V_S_CONSUME ORDER BY SNO;
```

SNO	SName	College	GoodsKinds	CostTotal
S01	李明	CS	1	10.50
S02	徐好	CS	2	15.10
S03	伍民	CS	1	3.50
S04	闵红	ACC	0	0.00
S05	张小红	ACC	2	31.40

结果分析：使用 COUNT(DISTINCT GoodsNO) 统计种类，数量乘当前商品售价计算消费额。样本没有保存历史成交价，因此不能把此值视为历史实付金额。未购买者消费额为 0。

6. 定义类别商品总数平均售价及最小库存视图 V_GOODS

操作 1：

```
CREATE VIEW V_GOODS AS
SELECT c.CategoryNO,c.CategoryName,COUNT(g.GoodsNO) AS GoodsCount,
CAST(AVG(g.SalePrice) AS DECIMAL(10,2)) AS AvgPrice,
MIN(g.Number) AS MinStock
FROM Category AS c LEFT JOIN Goods AS g ON g.CategoryNO=c.CategoryNO
GROUP BY c.CategoryNO,c.CategoryName;
```

语句执行成功。

操作 2：

```
SELECT * FROM V_GOODS ORDER BY CategoryNO;
```

CategoryNO	CategoryName	GoodsCount	AvgPrice	MinStock
CN001	咖啡	2	4.65	50
CN002	洗发水	1	19.80	65
CN005	床上用品	1	150.00	28

结果分析：商品总数在本题按商品种类条数 COUNT(GoodsNO) 理解；平均价格和最小库存按对应商品计算。

7. 定义供应商商品数库存和平均进价视图 V_SU_GOOD

操作 1：

```
CREATE VIEW V_SU_GOOD AS
SELECT s.SupplierNO,s.SupplierName,s.Number AS Phone,s.Address,
COUNT(g.GoodsNO) AS GoodsCount,SUM(g.Number) AS StockTotal,
CAST(AVG(g.InPrice) AS DECIMAL(10,2)) AS AvgInPrice
FROM Supplier AS s JOIN Goods AS g ON g.SupplierNO=s.SupplierNO
GROUP BY s.SupplierNO,s.SupplierName,s.Number,s.Address
HAVING AVG(g.InPrice)>10;
```

语句执行成功。

操作 2：

```
SELECT SupplierNO,SupplierName,GoodsCount,StockTotal,AvgInPrice
FROM V_SU_GOOD ORDER BY SupplierNO;
```

SupplierNO	SupplierName	GoodsCount	StockTotal	AvgInPrice
sup003	飞鹤食品	1	65	15.00
sup005	缙云日化	1	28	120.00

结果分析：按供应商聚合，平均进价超过 10 元者为飞鹤食品和缙云日化；同时保留联系方式，满足第 10 项查询。

8. 通过 V_COM 查询平均年龄十九以上的学院

```
SELECT * FROM V_COM WHERE AvgAge>19 ORDER BY College;
```

College	StudentCount	AvgAge
ACC	2	29.00
CS	3	28.33

结果分析：两个学院均满足条件。

9. 通过 V_S_CONSUME 查询消费总额超过二百的学生

```
SELECT * FROM V_S_CONSUME WHERE CostTotal>200 ORDER BY SNO;
```

语句执行成功。

结果分析：最大消费额为 31.40，返回空集。

10. 通过 V_SU_GOOD 查询库存总数小于五十的供应商

```
SELECT SupplierName,Phone,Address FROM V_SU_GOOD
WHERE StockTotal<50 ORDER BY SupplierNO;
```

SupplierName	Phone	Address
缙云日化	19648768903	重庆北碚

结果分析：返回缙云日化的名称、电话和地址。

11. 通过 V_MIS 插入宋江并查询

操作 1：

```
INSERT INTO V_MIS (SNO,SName,BirthYear,College,Major,WeiXin)
VALUES ('s09','宋江',2000,'CS','MIS','wx009');
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT SNO,SName,Major FROM V_MIS WHERE SNO='s09';
```

SNO	SName	Major
s09	宋江	MIS

结果分析：宋江专业为 MIS，插入 1 行且在视图中可见。题目未给出性别，原始恢复结构允许 Gender 为空，不推测性别。

12. 通过 V_MIS 插入周瑜并查询

操作 1：

```
INSERT INTO V_MIS (SNO,SName,BirthYear,College,Major,WeiXin)
VALUES ('s10','周瑜',1999,'CS','IT','wx010');
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT SNO,SName,Major FROM V_MIS WHERE SNO='s10';
```

语句执行成功。

操作 3：

```
SELECT SNO,SName,Major FROM Student WHERE SNO='s10';
```

SNO	SName	Major
s10	周瑜	IT

结果分析：没有 CHECK OPTION 时，IT 学生可通过可插入视图写入基表，但不满足 MIS 筛选，查询该视图看不到记录。

13. 通过 V_MIS 将 s01 的姓名改为李欢欢

操作 1：

```
UPDATE V_MIS SET SName='李欢欢' WHERE SNO='s01';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：0。

操作 2：

```
SELECT SNO,SName,Major FROM Student WHERE SNO='s01';
```

SNO	SName	Major
S01	李明	IT

结果分析：S01 为 IT 专业，视图不可见，因此更新成功但影响 0 行，基表仍为李明。

14. 通过 V_MIS 将 s02 的姓名改为徐慧慧

操作 1：

```
UPDATE V_MIS SET SName='徐慧慧' WHERE SNO='s02';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT SNO,SName FROM V_MIS WHERE SNO='s02';
```

SNO	SName
S02	徐慧慧

结果分析：S02 为 MIS，修改影响 1 行，基表和视图均反映新姓名。

15. 通过 V_SSG 将力神咖啡售价改为十元

操作 1：

```
UPDATE V_SSG SET SalePrice=10 WHERE GoodsName='力神咖啡';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：0。

操作 2：

```
SELECT GoodsName,SalePrice FROM V_SSG WHERE GoodsName='力神咖啡';
```

语句执行成功。

结果分析：商品表没有力神咖啡，更新影响 0 行。这不表示连接视图不能更新；另用事务内临时商品和销售记录验证，单独修改 Goods.SalePrice 可以成功，补充数据随后回滚。

16. 通过 V_MIS 删除 s09

操作 1：

```
DELETE FROM V_MIS WHERE SNO='s09';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：1。

操作 2：

```
SELECT SNO FROM Student WHERE SNO='s09';
```

语句执行成功。

结果分析：宋江为 MIS，可见且没有被销售记录引用，删除影响 1 行，基表也不再存在。

17. 通过 V_MIS 删除 s10

操作 1：

```
DELETE FROM V_MIS WHERE SNO='s10';
SELECT ROW_COUNT() AS affected_rows;
```

实际影响行数：0。

操作 2：

```
SELECT SNO,SName,Major FROM Student WHERE SNO='s10';
```

SNO	SName	Major
s10	周瑜	IT

结果分析：周瑜属于 IT，不能从 MIS 视图匹配，因此删除影响 0 行，基表仍保留该学生。

补充验证与最终状态

除以上任务，还保存了 8 项补充验证记录。临时补充数据在事务中回滚，没有写入原始样本作为题目答案。

验证项目	实际结果
临时力神咖啡通过连接视图改为 10 元	通过
临时商品和销售记录回滚	通过
创建带 CHECK OPTION 的对照视图	通过
对照视图拒绝插入 IT 学生	ERROR 1369
移除验证用对照视图	通过
聚合视图拒绝更新	ERROR 1288
读取各视图可更新性标志	通过
最终学生名单符合可见性和删除结果	通过

TABLE_NAME	IS_UPDATABLE
v_age	YES
v_com	NO
v_goods	NO
v_mis	YES
v_s_consume	NO
v_ssg	YES
v_su_good	NO

连接视图正例的实际执行 SQL 如下，临时样本只存在于本事务中：

```
START TRANSACTION;
INSERT INTO Goods VALUES
('GNTEST','sup001','CN001','力神咖啡',5,8,1,'2026-10-11');
INSERT INTO SaleBill VALUES ('GNTEST','S01','2026-10-11',1);
UPDATE V_SSG SET SalePrice=10 WHERE GoodsName='力神咖啡';
SELECT SalePrice FROM Goods WHERE GoodsNO='GNTEST';
ROLLBACK;
```

最终表行数：

关系表	行数
Category	3
Supplier	5
Student	6
Goods	4
SaleBill	6

原始 SQL、逐题执行日志、结构化汇总与最终数据库导出已保存，可根据记录复核结果。

七、实验思考

1. 视图定义能否使用查询语句的任意子句？

MySQL 视图可以使用连接、分组和聚合等查询结构，但并非任意查询上下文都适用；定义不能引用用户变量或临时表。聚合、DISTINCT、GROUP BY、UNION 等结构会影响可更新性。视图内部的 ORDER BY 不能代替调用者的排序要求，本实验均在最终查询中明确排序。

2. 查询视图和基本表有什么区别？

普通视图保存的是查询定义，本身不保存本次查询的结果数据。数据库对视图定义进行合并或物化后执行，数据仍来自基本表。基本表变化会影响之后的视图查询结果。

3. 哪些视图通常可以更新？

能明确对应基本表记录的简单单表视图通常可更新。MySQL 某些可合并的连接视图也允许更新其中一个基本表，本次 V_SSG 的商品售价更新已通过正例验证。聚合视图 V_COM 不可更新，实测返回 ERROR 1288；不能笼统地把所有连接视图认定为只读。

4. 使用视图有什么好处？

视图可封装常用连接和统计，简化查询，并为不同使用者暴露需要的列和记录。WITH CHECK OPTION 可防止通过视图写入不满足筛选条件的记录；是否使用应由业务要求决定，不能把筛选条件与写入检查混为一谈。

八、实验总结

本次完成 7 个视图定义及全部查询和修改任务。实验表明，视图可见性、写入检查和视图可更新性是不同问题：不满足 MIS 条件的学生可写入基表却不出现在视图中，聚合视图拒绝更新，而满足条件的连接视图可以更新单一基本表。